

传输层协议

计算机网络 期末复习2

ywy_c_asm

计算学部金牌讲师团

目录

- 1. 传输层概述
- 2. UDP协议
- 3. 停等协议(rdt3.0)
- 4. GBN协议
- 5. SR协议
- 6. TCP协议
- 7. TCP动态控制机制
- 8. 一些TCP窗口题
- 附录：第3章你或许应该知道的名词缩写

1. 传输层概述

03. 传输层为 () 之间提供逻辑通信。

A. 主机

☒ B. 进程

C. 路由器

D. 操作系统

08. 在 () 范围内的端口号被称为“熟知端口号”并限制使用。这就意味着这些端口号是为常用的应用层协议如 FTP、HTTP 等保留的。

A. 0~127

B. 0~255

C. 0~511

☒ D. 0~1023

• 传输层为进程提供逻辑通信，将从应用进程接受的报文转换为报文段进行端到端传输，传输层的TCP/UDP协议都使用网络层提供的IP协议(不可靠的尽力而为交付)。

• 进程通过1个或多个套接字使用传输层服务进行数据传输。传输层从不同的套接字中接收报文，加上标识套接字的首部信息，生成报文段交付给网络层，这是传输层的多路复用。传输层还需要将从网络层接收的报文段数据通过其首部信息交付给正确的套接字，这是传输层的多路分解。

3. 在 Internet 中，复用/分解服务只需在传输层实现。 (☒)

• 端口号(16位, 0~65535)用于对套接字进行标识，0~1023为周知端口号，不能随便使用。报文段首部记录了源和目的的IP地址+端口号。TCP协议和UDP协议的端口号以及套接字是独立的：

传输层在进行多路分解时，会先根据协议字段判断交付给TCP套接字还是UDP套接字

• UDP是无连接协议，套接字用二元组(目的IP, 目的端口号)标识，来自不同的源的数据可以被送到同一个目的套接字，接收方本来不知道源，收到数据后可以顺便得知报文段首部记录的源IP+源端口号。

• TCP是有连接协议，套接字用四元组(源IP, 源端口号, 目的IP, 目的端口号)标识，来自不同的源的数据只能通过各自的TCP连接被送到不同的目的套接字，而这些套接字可使用相同端口号，例如Web服务器可并行维护多个TCP连接，它们都使用80端口，报文段通过源地址来决定被送到哪个套接字。

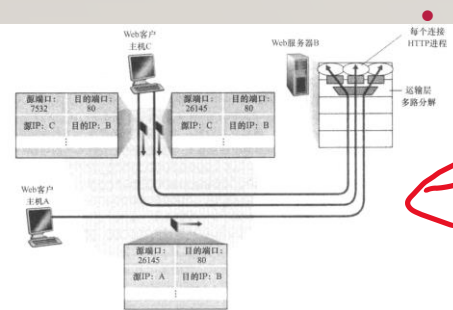


图 3-5 两个客户使用相同的端口号 (80) 与同一个 Web 服务器应用通信

10. 关于 TCP 和 UDP 端口的下列说法中，正确的是 ()。

☒ A. TCP 和 UDP 分别拥有自己的端口号，它们互不干扰，可以共存于同一台主机

B. TCP 和 UDP 分别拥有自己的端口号，但它们不能共存于同一台主机

C. TCP 和 UDP 的端口没有本质区别，但它们不能共存于同一台主机

D. 当一个 TCP 连接建立时，它们互不干扰，不能共存于同一台主机

12. 【2018 统考真题】UDP 协议实现分用时所依据的头部字段是 ()。

☒ A. 源端口号

☒ B. 目的端口号

C. 长度

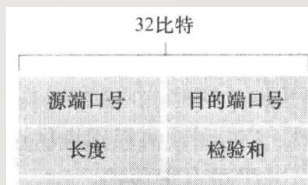
D. 校验和

2.UDP协议

11. 【2014 统考真题】下列关于 UDP 协议的叙述中, 正确的是 ()。

- ✓ I. 提供无连接服务
 ✓ II. 提供复用/分用服务
 III. 通过差错校验, 保障可靠数据传输
 A. 仅 I B. 仅 I、II

- **UDP协议**无连接无状态，传输不可靠，仅提供基本的复用/分用和少量差错检测，报文段首部开销少(基本就是对IP数据报的简单封装)，无拥塞控制，应用可以更精细地控制发送时间和速率。



- UDP报文段首部**8字节**， 4个16位字段：

- 源端口号和目的端口号实现了基本的多路分解。

- **长度**字段表示整个UDP报文段的字节数，包括**首部+数据载荷**，最小值为8。

- **检验和**字段的计算：先在末尾补0使得报文长度为偶数字节，将检验和字段初始化为0，每两个字节(16位)进行16位累加，若中间有进位则将其重新**加到最低位**(回卷)，将最终的结果**按位取反**即得到检验和。接收方以同样方式对报文段进行求和，应该得到**全1**的结果即视为正确。若检验和错误，UDP无法对其进行恢复，只能丢弃或警告。注意，IP数据报首部中的源IP地址和目的IP地址也要被算入检验和！**称之为“伪首部”，包括协议字段**

04. UDP 数据报中的长度字段 ()。

- A. 不记录数据的长度
B. 只记录首部的长度
C. 只记录数据部分的长度
D. 包括首部和数据部分的长度

不包含检验和字段的和是 x ，检验和字段是 $\sim x$ ，最终的和就应该是全1

10. 在计算 UDP 校验和时,封装 UDP 报文段的 IP 数据报首部某些字段也会参与计算,这些字段是()

- I. 源 IP 地址 II. 目的 IP 地址 III. 协议 IV. TTL

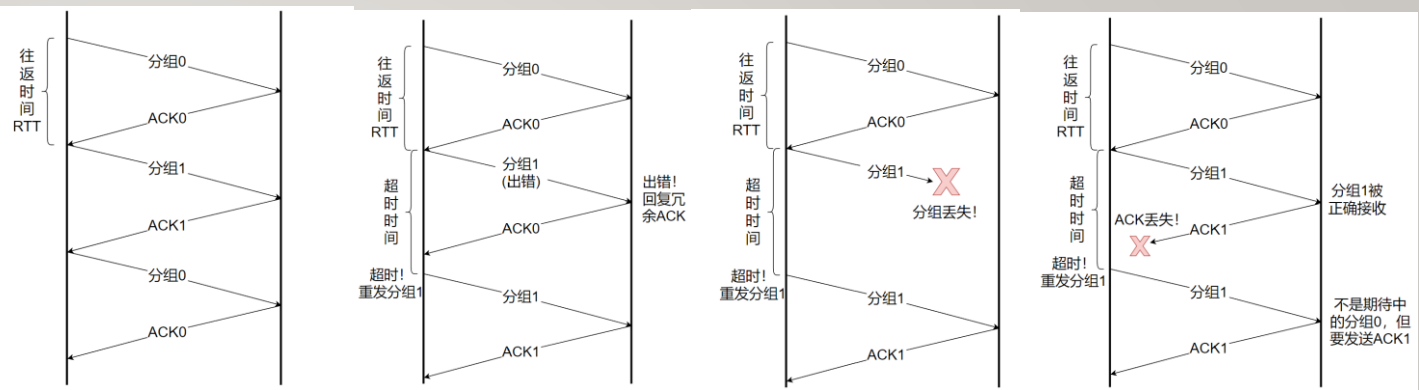
07. 接收端收到有差错的 UDP 用户数据时的处理方式是 ()。

- A. 丢弃 B. 请求重传 C. 差错校正 D. 忽略差错

3. 停等协议(rdt3.0)

11. 对于停等协议(rdt3.0), 若接收方正确接收了 1 号分组, 且发送了对 1 号分组的确认 ACK1, 之后又收到了 1 号分组, 则接收方针对该分组的正确操作是()

- ✓ 丢弃该分组, 发送 ACK1
✗ 丢弃该分组, 发送 ACK0
✗ 丢弃该分组, 无需发送 ACK
D. 接受该分组, 并向上层提交数据, 发送 ACK1
- 发送方会永远超时



- **停等协议**实现了低效率的可靠数据传输, 所有分组都以0/1交替编号, 发送方每发送一个分组都要等待接收方的确认, 若超时则重发分组。在数据传输过程中会出现**比特差错**或**分组丢失**。

- **发送方行为**: 从上层接收数据, 创建编号为*i*的分组, 将其发送并设置定时器。若收到ACK(*i*)则说明分组接收成功, 发送下一个分组。若超时则重新发送并重置定时器。

02. 在简单的停止等待协议中, 当帧出现丢失时, 发送端会永远等待下去, 解决这种死锁现象的办法是()。
A. 差错校验 B. 帧序号 C. NAK 机制 D. 超时机制

- **接收方行为**: 一直等待接收分组*i*, 若正确接收到分组*i*则交付上层并回复ACK(*i*), 若收到的分组有差错, 则回复一个ACK(!*i*), 若分组*i*是上一个接收的分组(**冗余分组**)则也要回复ACK(*i*)。

无论如何, ACK(*i*)都表示上一个被正确接收的编号为*i*

冗余ACK的作用类似NAK

这可能是因为上个分组本来正确接收但ACK丢失, 发送方又重发了一遍

- **信道利用率**为一个发送周期内发送方真正用于数据传输的时间的占比, 理想状况下停等协议的发送周期为**分组传输时延+单向传播时延+ACK传输时延+单向传播时延**。数据传输时间用带宽算。

18. 【2020 统考真题】假设主机甲采用停-等协议向主机乙发送数据帧, 数据帧长与确认帧长均为 1000B, 数据传输速率是 10kb/s, 单向传播延时是 200ms。则甲的**最大信道利用率**为()。

最理想状况, 无丢失无差错无超时

- A. 80% B. 66.7% C. 44.4% D. 40%

16. 【2018 统考真题】主机甲采用停止-等待协议向主机乙发送数据, 数据传输速率是 3kb/s, 单向传播时延是 200ms, 忽略确认帧的传输时延。当信道利用率等于 40%时, 数据帧的长度为()。

- A. 240 比特 B. 400 比特 C. 480 比特 D. 800 比特

$$\text{分组传输时间} = \frac{1000\text{B} \times 8}{10\text{kb/s}} = 800\text{ms}$$

$$\text{传输周期} = 800\text{ms} + 200\text{ms} + 800\text{ms} + 200\text{ms} = 2000\text{ms}$$

$$\text{理想状况下最大信道利用率} = \frac{800\text{ms}}{2000\text{ms}} = 40\%$$

$$\text{传输周期} = \frac{200\text{ms} + 200\text{ms}}{1 - 40\%} = 666.67\text{ms}$$

$$\text{分组传输时间} = 666.67\text{ms} - 200\text{ms} - 200\text{ms} = 266.67\text{ms}$$

$$\text{分组长度} = 266.67\text{ms} \times 3\text{kb/s} = 800\text{bit}$$

4. GBN协议

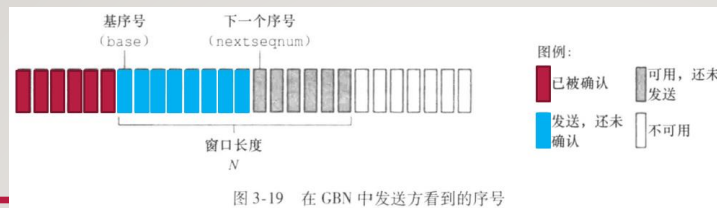


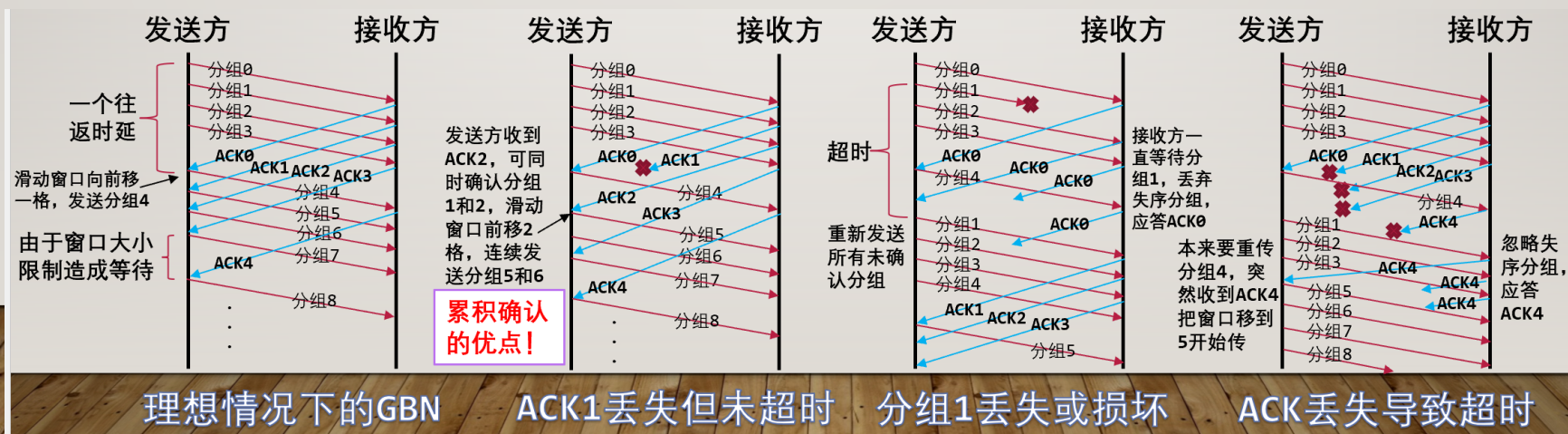
图 3-19 在 GBN 中发送方看到的序号

04. 采用后退 N 帧协议方式, 发送方已经发送了编号 0~6 的帧。计时器超时, 只收到了对 1、3 和 5 号帧的确认, 发送方需要重传的帧的数目是 ()。

A. 1 B. 2 C. 5 D. 6

- GBN协议增大分组的编号范围, 对发送方设置一个大小为 N 的滑动窗口, 可以连续发送分组。
接收方没有缓存窗口
- 接收方行为**: 一直等待接收下一个分组, 如果收到的分组不是期望的那个(失序或出错)则直接扔掉。接收方应答采取累积确认方式, 无论如何都要回复ACK(i)表示上个正确接收编号是 i 。
累积确认可以降低ACK丢失的影响, 即便一个ACK丢失, 只要后面的ACK收到, 仍无影响
- 发送方行为**: 发送方不断从上层接收分组将其发送并缓存在窗口里, 直至窗口满为止, 窗口内缓存的都是已发送未确认的分组(限制不超过 N 个), 设窗口内第一个编号为base, 为这个最早发送的未确认分组设置1个定时器, 若超时则重发所有已发送未确认分组(Go Back至多 N 步), 若收到一个编号在窗口内的ACK(x), 说明base~ x 都已被正确接收, 将窗口起点推移到 $x+1$ 。
此时一般可以发送新的分组

$N=4$ 的典型例子:



理想情况下的GBN

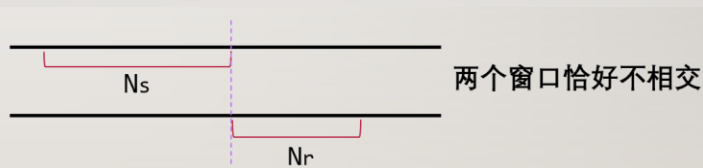
ACK1丢失但未超时

分组1丢失或损坏

ACK丢失导致超时

4. GBN协议-计算

窗口位置极端情况：



窗口大小不满足上界的反例：



- 在现实中编号空间是有限的，一般是**模 2^k 循环**使用的 $(0, 1, \dots, 2^k - 1, 0, 1, 2, \dots)$ 。无论是哪种滑动窗口协议，接收方窗口(对于GBN来说大小为1)起始位置一定**不可能在发送方窗口前面**(发送方窗口前面的分组都已被确认)。为保证任何情况下两个窗口内编号不冲突，必须有 $N_s + N_r \leq 2^k$ ，对于GBN协议来说就是 $N \leq 2^k - 1$ ，这是上界。
- 理想情况下GBN的发送周期为开始连续发送N个分组的时间到开始连续发送下N个分组的时间(即收到第一个分组的ACK)，这个发送周期和停等协议一样，仍为**分组传输时延+单向传播时延+ACK传输时延+单向传播时延**，但在这个周期里传输了N个分组，因此信道利用率高。

17. 【2019 统考真题】对于滑动窗口协议，若分组序号采用 3 比特编号，发送窗口大小为 5，则接收窗口最大是 ()。

- A. 2 **B. 3** C. 4 D. 5

12. 若主机甲使用 GBN 协议向主机乙发送分组，分组长度 500B，采用 4bit 编号，数据传输速率是10Mbps ($M = 10^6$)，单向传播时延为 5.8ms，忽略 ACK 传输时延，则甲的最大信道利用率约为 ()

- A. 25% **B. 50%** C. 75% D. 100%

最大窗口大小 $N = 2^4 - 1 = 15$

单个分组传输时延 = $\frac{500B \times 8}{10Mbps} = 0.4ms$

发送周期 = $0.4ms + 5.8ms + 5.8ms = 12ms$

信道利用率 = $\frac{15 \times 0.4ms}{12ms} = 50\%$

13. 【2012 统考真题】两台主机之间的数据链路层采用后退 N 帧协议 (GBN) 传输数据，数据传输速率为 16 kb/s，单向传播时延为 270ms，数据帧长度范围是 128 ~ 512 字节，接收方总是以与数据帧等长的帧进行确认。为使信道利用率达到最高，帧序号的比特数至少为 ()。要找一个足够大的窗口，使得无论分组多大，信道利用率都能最高

- A. 5 **B. 4** C. 3 D. 2

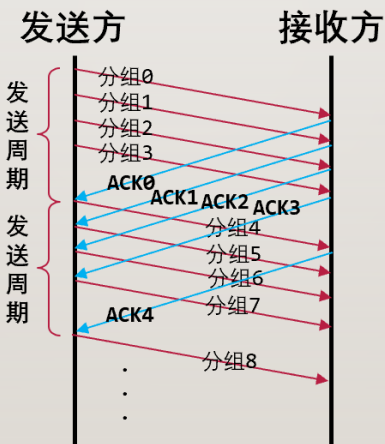
取128B的分组，它所需窗口大于512B的

单个分组传输时延 = $\frac{128B \times 8}{16kbps} = 64ms$

发送周期 = $64ms + 270ms + 64ms + 270ms = 668ms$

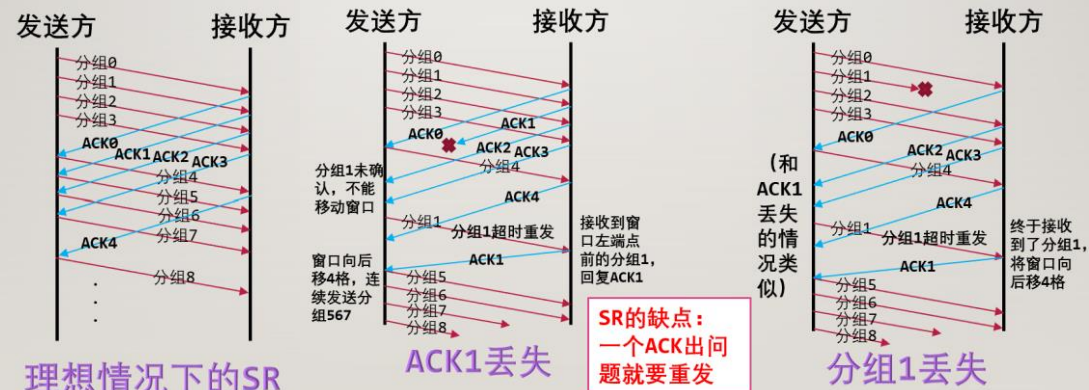
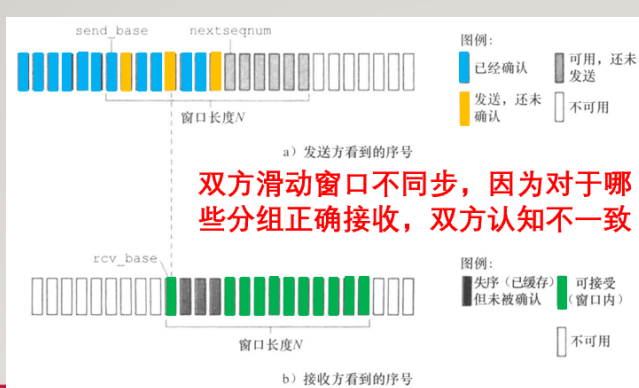
信道利用率最大时，发送周期必须被传输时延充满，应该发送至少

$\frac{668ms}{64ms} = 10.4$ 个，窗口大小至少11，需要4bit

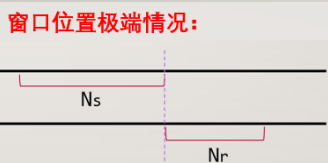


理想情况下的GBN

5. SR协议



- **SR协议**令接收方也设置一个 N_r 大小窗口缓存失序分组并**逐个确认**，分组可以乱序到达。
 但对上层的交付是按序的
- **接收方行为**：设窗口左端点为base，收到一个编号为i的正确分组，若i在窗口内($i \in [base, base + N_r - 1]$)，则将分组缓存并回复ACK(i)。若i在窗口之前并且距离不超过N($i \in [base - N_s, base - 1]$)，说明这是发送方因ACK丢失而重发的**冗余分组**，丢弃分组但也要回复ACK(i)。其余情况下(分组损坏或编号无效)直接丢弃，也不回复ACK。若base分组被缓存，那么接收方会推移窗口，将窗口左端点被缓存的分组交付上层，直到左端点未被缓存为止。
- **发送方行为**：发送方仍在窗口内缓存已发送未确认的分组，但允许窗口内有一些分组已经得到确认(当收到这个分组的ACK时)。若窗口左端点得到确认，往后推移窗口直到窗口左端点未确认为止。对**每一个**未确认分组都要设置定时器，当某个分组超时仅重发它即可。



由于理想情况下SR和GBN过程类似，计算题(信道利用率)一般直接使用GBN

发送方窗口更大则发的某些分组无法被接收，接收方窗口更大则没有意义

- SR协议的窗口大小事实上应当满足 $N_s = N_r = N$ ，由 $N_s + N_r \leq 2^k$ ，得到上界 $N \leq 2^{k-1}$ 。

12. 【2011 统考真题】数据链路层采用选择重传协议(SR)传输数据，发送方已发送 0~3 号数据帧，现已收到 1 号帧的确认，而 0、2 号帧依次超时，则此时需要重传的帧数是 ()。

- A. 1 B. 2 C. 3 D. 4

09. 对无序接收的滑动窗口协议，若序号位数为 n ，则发送窗口最大尺寸为 ()。

- A. $2^n - 1$ B. $2n$ C. $2n - 1$ D. 2^{n-1}

几个典型例子:

6. TCP协议

40. 【2021 统考真题】若大小为 12B 的应用层数据分别通过 1 个 UDP 数据报和 1 个 TCP 段传输, 则该 UDP 数据报和 TCP 段实现的有效载荷(应用层数据)最大传输效率分别是 (). $12/(12+8)=60\%, 12/(12+20)=37.5\%$
A. 37.5%, 16.7% B. 37.5%, 37.5% C. 60.0%, 16.7% ☒ D. 60.0%, 37.5%

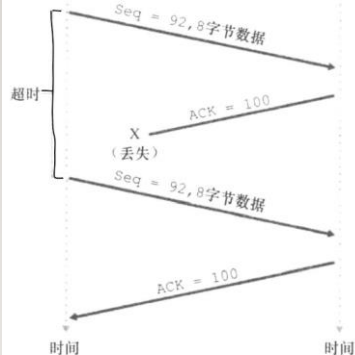


图 3-34 由于确认丢失而重传

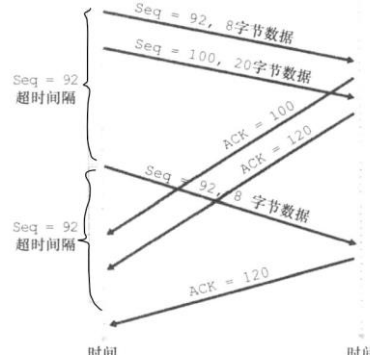


图 3-35 报文段 100 没有重传

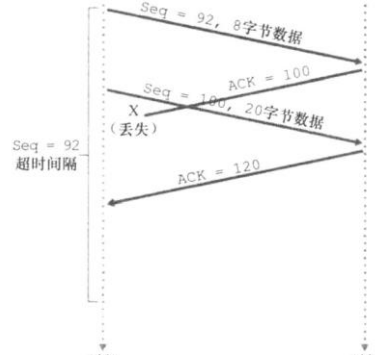


图 3-36 累积确认避免了第一个报文段的重传

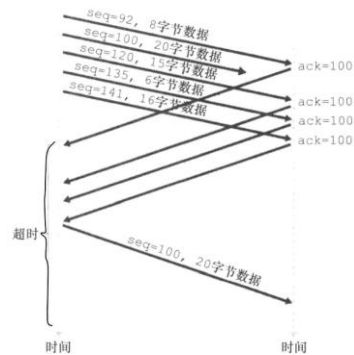


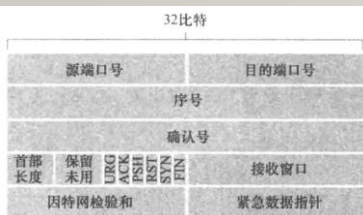
图 3-37 快速重传: 在某报文段的定时器过期之前重传丢失的报文段

- TCP协议是面向连接的, 连接状态保持在端系统中, 支持全双工点对点传输, 每个报文段最多能携带 **MSS** 字节(最大报文段长度)的有效数据, ACK 可被捎带在数据报文段中。

03. 下列 () 不是 TCP 服务的特点。

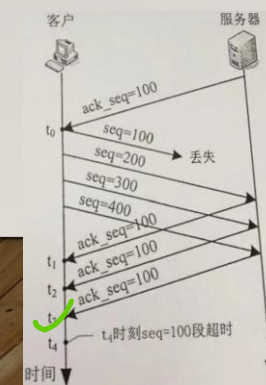
A. 字节流 B. 全双工 C. 可靠 ☒ D. 支持广播

- TCP 报文的首部一般为 **20 字节**, 包括源/目的端口号, 32 位序号, 32 位 ACK 号, 16 位检验和, 16 位接收窗口大小(流量控制)以及标志位(确认号字段有效标志 **ACK**, 连接建立标志 **SYN**, 连接结束标志 **FIN**)。

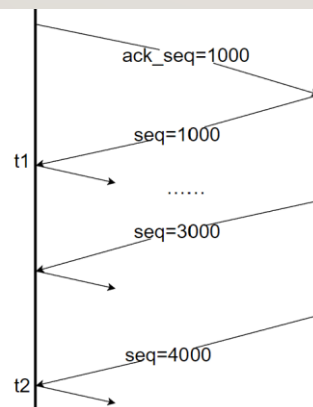


TCP 的序号建立在 **字节** 流上, 是最后一个被成功发送的字节序号+1(即该报文有效数据首字节序号), ACK 号采取 **累积确认**, 表示接收方期望接收的 **下一个** 字节的序号。即接收窗口内第一个未收到分组这和GBN稍有不同

- TCP 发送方仅为窗口内第一个已发送未确认分组设置 **单一** 计时器, 超时后仅重传该分组, 收到 ACK 后可以往后推移窗口。接收方可以在窗口内缓存失序分组。发送方采取 **快速重传** 机制, 当接收到对窗口前一个分组的 **3 个冗余** ACK(即实际上收到 4 个)后不等超时发生就直接重传第一个分组。



14. 如左图所示, 若 TCP 支持快速重传, 则客户重新发送丢失的 seq=100 的报文段的时刻是 ()



6. 如左图所示, 服务器均以 MSS=1000B 向客户发送 TCP 报文段, 则客户在 t1 和 t2 时刻分别发送给服务器的确认序号分别为 ()

A. 1000, 4000 B. 2000, 2000 C. 3000, 4000 ☒ D. 2000, 5000

30. 【2011 统考真题】主机甲与主机乙之间已建立一个 TCP 连接, 主机甲向主机乙发送了 3 个连续的 TCP 段, 分别包含 300B、400B 和 500B 的有效载荷, 第 3 个段的序号为 900。若主机乙仅正确接收到第 1 个段和第 3 个段, 则主机乙发送给主机甲的确认序号是 ()。期望接收首字节编号 500 的第二个段

A. 300 ☒ B. 500 C. 1200 D. 1400

6. TCP协议-连接建立与关闭

注意这些握手的序号与确认号:

这是为了防止网络中来自原先连接的残存报文段的干扰

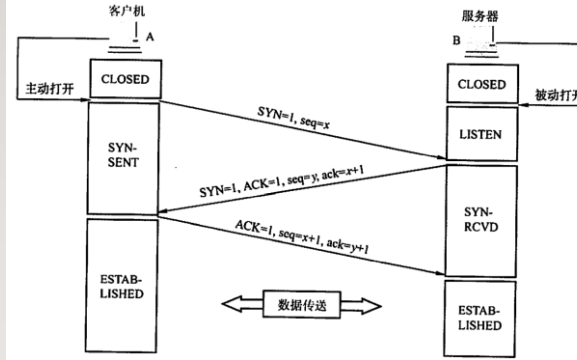


图 5.7 用“三次握手”建立 TCP 连接

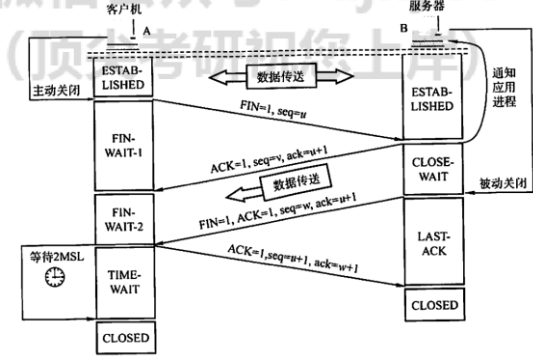


图 5.8 用“四次握手”释放 TCP 连接

- TCP连接建立需要三次握手，客户先**随机**选择初始编号`client_isn`，向服务器发送不带有有效数据的**SYN=1**报文段，服务器收到后为连接分配缓存，随机选择初始编号`server_isn`，向客户应答**SYN=1**的SYNACK报文段(应答号为`client_isn+1`，因为前两次握手的报文段**被规定消耗一个序号**)，客户收到后为连接分配缓存，向服务器发送**SYN=0**的可以携带数据的报文段。

尽管它们不携带数据

- TCP连接关闭需要四次握手，当A单方面希望关闭连接时，发送**FIN=1**报文段，B**被动**接收到时，会回复一个普通ACK，并进行关闭前的一些准备工作(例如通知应用进程以及向A发送一些数据)。当B准备好关闭连接时，会向A发送**FIN=1**报文段，A收到后会回复普通ACK，等待一段时间后若无重发的FIN报文段则A正式关闭连接。B在收到ACK后正式关闭连接。

FIN报文段也被规定消耗一个序号

此时A不能再发数据了

15. 若主机甲主动发起一个与主机乙的 TCP 连接, 甲乙选择的初始序号分别为 2019 和 2046, 则第三次握手 TCP 报文段的确认号是()

- A. 2019 B. 2020 C. 2046 **D. 2047**

7. 若客户与服务器建立 TCP 连接时, 第三次握手的 TCP 段的序号为 1020, 当客户向服务器发送了 1000B 的数据后, 首先请求与服务器断开连接, 则客户收到的第二次握手 TCP 段的确认序号是()

- A. 2019 B. 2020 **C. 2021** D. 2022

29. 【2011 统考真题】主机甲向主机乙发送一个 (SYN = 1, seq = 11220) 的 TCP 段, 期望与主机乙建立 TCP 连接, 若主机乙接受该连接请求, 则主机乙向主机甲发送的正确的 TCP 段可能是()。

- A. (SYN = 0, ACK = 0, seq = 11221, ack = 11221)
B. (SYN = 1, ACK = 1, seq = 11220, ack = 11220)
C. (SYN = 1, ACK = 1, seq = 11221, ack = 11221)
D. (SYN = 0, ACK = 0, seq = 11220, ack = 11220)

38. 【2020 统考真题】若主机甲与主机乙建立 TCP 连接时, 发送的 SYN 段中的序号为 1000, 在断开连接时, 甲发送给乙的 FIN 段中的序号为 5001, 则在无任何重传的情况下, 甲向乙已经发送的应用层数据的字节数为()。 **从第三次握手开始, 序号1001**

- A. 4002 B. 4001 **C. 4000** D. 3999

7. TCP动态控制机制

05. 已知当前 TCP 连接的 RTT 值为 35ms, 连续收到 3 个确认报文段, 它们比相应的数据报文段的发送时间滞后了 27ms、30ms 与 21ms。设 $\alpha = 0.2$ 。计算第三个确认报文段到达后的新的 RTT 估计值。 SampleRTT EstimatedRTT_新 = 30.4ms

- TCP在传输过程中对分组的实际往返时间RTT进行记录采样, 称为SampleRTT, 通过采样值使用指数加权移动平均得出RTT的估计值EstimatedRTT以及RTT的波动程度DevRTT, 有:

$$EstimatedRTT_{\text{新}} = (1 - \alpha)EstimatedRTT_{\text{旧}} + \alpha \times SampleRTT$$

$$DevRTT_{\text{新}} = (1 - \beta)DevRTT_{\text{旧}} + \beta \times |SampleRTT - EstimatedRTT|$$

$$TimeoutInterval = EstimatedRTT + 4 \times DevRTT$$

根据实际网络波动
态设置超时时间阈值

注意: TCP中窗口
大小都是字节数表
示, 而不是分组数!

- TCP发送方的窗口大小同时受到流量控制和拥塞控制的限制, 对于流量控制, 令rwnd为接收方缓存剩余空间, 发送方的窗口大小(已发送未确认字节数)不得超过rwnd, rwnd通过TCP报文段首部字段传递给发送方。这样发送方就不至于发送数据过快使得接收方缓存溢出。
称为“接收窗口”
- 对于拥塞控制, 发送方维护“拥塞窗口”变量cwnd, 则最终的发送窗口大小 = $\min\{rwnd, cwnd\}$ 。

09. 在 TCP 协议中, 发送方的窗口大小取决于 ()。

- A. 仅接收方允许的窗口
- B. 接收方允许的窗口和发送方允许的窗口
- ☒ C. 接收方允许的窗口和拥塞窗口
- D. 发送方允许的窗口和拥塞窗口

28. 【2010 统考真题】主机甲和主机乙之间已建立一个 TCP 连接, TCP 最大段长为 1000B。若主机甲的当前拥塞窗口为 4000B, 在主机甲向主机乙连续发送两个最大段后, 成功收到主机乙发送的第一个段的确认段, 确认段中通告的接收窗口大小为 2000B, 则此时主机甲还可以向主机乙发送的最大字节数是 ()。 拥塞窗口足够大, 此时由接收窗口决定

- ☒ A. 1000
- B. 2000
- C. 3000
- D. 4000

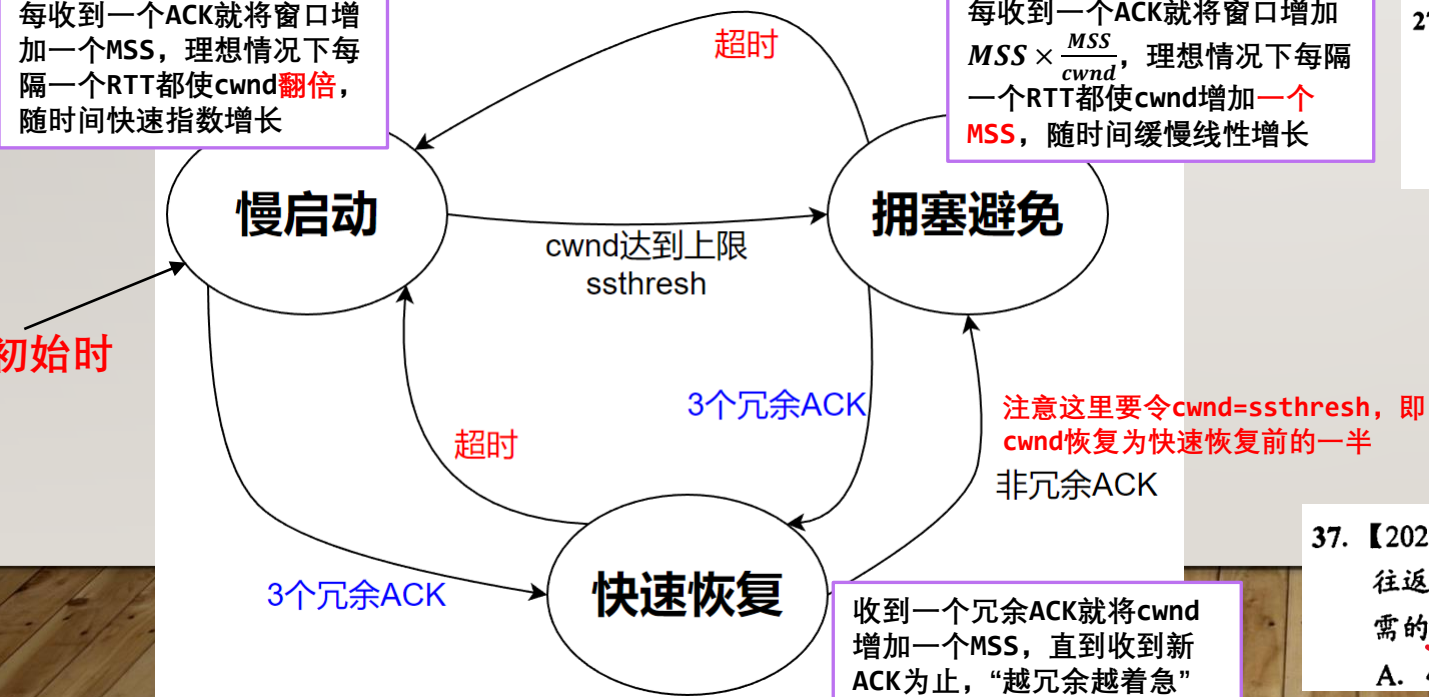
7. TCP动态控制机制-拥塞窗口的维护(重点)

- 发送方具有3个状态：慢启动，拥塞避免，快速恢复。发送方对于拥塞的判断由超时和收到3个冗余ACK两种重传事件引起。发送方还维护一个慢启动阈值 $ssthresh$ ，无论如何，只要出现任一重传事件，就令 $ssthresh = \frac{cwnd}{2}$ 。只要出现超时事件，就要令 $cwnd = 1MSS$ 并进入慢启动状态。(非快速恢复时)只要出现冗余ACK就要进入快速恢复状态。

实际解题时应考虑 $cwnd$ 随时间(RTT)的变化，最好别想随分组个数的变化

每收到一个ACK就将窗口增加一个MSS，理想情况下每隔一个RTT都使 $cwnd$ 翻倍，随时间快速指数增长

每收到一个ACK就将窗口增加 $MSS \times \frac{MSS}{cwnd}$ ，理想情况下每隔一个RTT都使 $cwnd$ 增加一个MSS，随时间缓慢线性增长



27. 【2009 统考真题】一个 TCP 连接总以 1KB 的最大段长发送 TCP 段，发送方有足够多的数据要发送，当拥塞窗口为 16KB 时发生了超时，如果接下来的 4 个 RTT 时间内的 TCP 段的传输都是成功的，那么当第 4 个 RTT 时间内发送的所有 TCP 段都得到肯定应答时，拥塞窗口大小是 ()。

A. 7KB B. 8KB C. 9KB D. 16KB

经过前3个RTT的快速恢复达到上限8KB，最后一个RTT拥塞避免，增长1KB

32. 【2014 统考真题】主机甲和乙建立了 TCP 连接，甲始终以 $MSS = 1KB$ 大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10KB 的确认段。若甲在 t 时刻发生超时的时拥塞窗口为 8KB，则从 t 时刻起，不再发生超时的情况下，经过 10 个 RTT 后，甲的发送窗口是 ()。

A. 10KB B. 12KB C. 14KB D. 15KB

慢启动上限4KB， $cwnd=1,2,4,5,6,7,8,9,10,11,12$ ，与10KB取min!!!

37. 【2020 统考真题】若主机甲与主机乙已建立一条 TCP 连接，最大段长 (MSS) 为 1KB，往返时间 (RTT) 为 2ms，则在不出出现拥塞的前提下，拥塞窗口从 8KB 增长到 32KB 所需的最长时间是 ()。

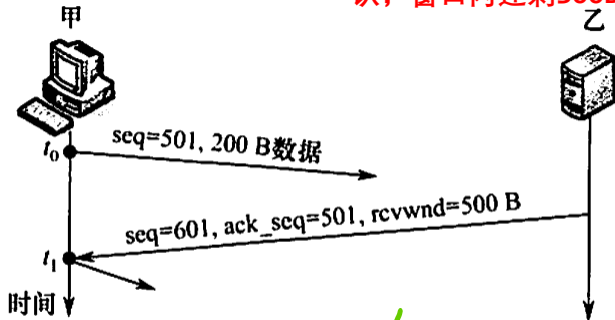
A. 4ms 线性增长 B. 8ms C. 24ms D. 48ms

8. 一些TCP窗口题

33. 【2015 统考真题】主机甲和主机乙新建一个 TCP 连接，甲的拥塞控制初始阈值为 32KB，甲向乙始终以 $MSS = 1KB$ 大小的段发送数据，并一直有数据发送；乙为该连接分配 16KB 接收缓存，并对每个数据段进行确认，忽略段传输延迟。若乙收到的数据全部存入缓存，不被取走，则甲从连接建立成功时刻起，未出现发送超时的情况下，经过 4 个 RTT 后，甲的发送窗口是 ()。 经过4个RTT后发送了1+2+4+8=15KB，接收方还剩1KB，即rwnd
- A. 1KB B. 8KB C. 16KB D. 32KB

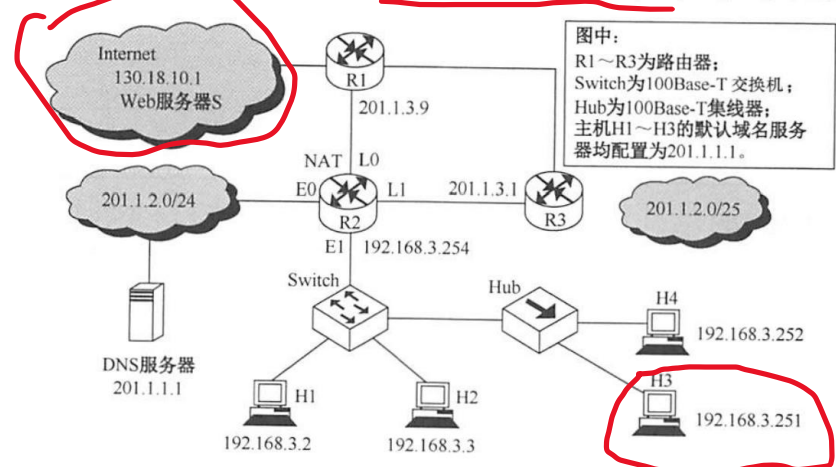
34. 【2017 统考真题】若甲向乙发起一个 TCP 连接，最大段长 $MSS = 1KB$ ， $RTT = 5ms$ ，乙开辟的接收缓存为 64 KB，则甲从连接建立成功至发送窗口达到 32KB，需经过的时间至少是 ()。 没说sssthresh就一直慢启动，经过5个RTT后cwnd翻倍到32KB，此时发送了1+2+4+8+16=31KB，rwnd为33KB，则此时发送窗口为32KB
- A. 25ms B. 30ms C. 160ms D. 165ms

41. 【2021 统考真题】设主机甲通过 TCP 向主机乙发送数据，部分过程如下图所示。甲在 t_0 时刻发送一个序号 $seq = 501$ 、封装 200B 数据的段，在 t_1 时刻收到乙发送的序号 $seq = 601$ 、确认序号 $ack_seq = 501$ 、接收窗口 $rcvwnd = 500B$ 的段，则甲在未收到新的确认段之前，可以继续向乙发送的数据序号范围是 ()。 大小200B的段[501,700]已发送未确认，窗口内还剩300B，即[701,1000]



- A. 501 ~ 1000 B. 601 ~ 1100 C. 701 ~ 1000 D. 801 ~ 1100

16. 【2016 统考真题】假设下图中的 H3 访问 Web 服务器 S 时，S 为新建的 TCP 连接分配了 20KB ($K = 1024$) 的接收缓存，最大段长 $MSS = 1KB$ ，平均往返时间 $RTT = 200ms$ 。H3 建立连接时的初始序号为 100，且持续以 MSS 大小的段向 S 发送数据，拥塞窗口初始阈值为 32KB；S 对收到的每个段进行确认，并通告新的接收窗口。假定 TCP 连接建立完成后，S 端的 TCP 接收缓存仅有数据存入而无数据取出。请回答下列问题：



- 在 TCP 连接建立过程中，H3 收到的 S 发送过来的第二次握手 TCP 段的 SYN 和 ACK 标志位的值分别是多少？确认序号是多少？ SYN=1, ACK=1, ACK号=100+1=101
- H3 收到的第 8 个确认段所通告的接收窗口是多少？此时 H3 的拥塞窗口变为多少？H3 的发送窗口变为多少？ rwnd=20-8=12KB, cwnd=1+8=9KB, 发送窗口=9KB
- H3 的发送窗口等于 0 时，下一个待发送的数据段序号是多少？H3 从发送第 1 个数据段到发送窗口等于 0 时刻为止，平均数据传输速率是多少？(忽略段的传输延时。)
- 若 H3 与 S 之间通信已经结束，在 t 时刻 H3 请求断开该连接，则从 t 时刻起，S 释放该连接的最短时间是多少？ 100ms后S收到FIN，发送FINACK，然后立刻发送自己的FIN，200ms后S收到FINACK，结束连接，最少共300ms

当H3发送窗口为0时，发送了20KB(20个段)，下一个序号为 $101+20*1024=20581$ ，接下来几个RTT中，每个RTT开始时都发送发送窗口大小的段数，每个RTT结束时根据ACK更新发送窗口大小，
 $(wnd, rwnd, cwnd) = [(1, 20, 1), (2, 19, 2), (4, 17, 4), (8, 13, 8), (5, 5, 16), (0, 0, 21)]$ ，
 因此过了 $5RTT=1000ms$ 后发送窗口变为0，平均数据传输速率= $\frac{20KB \times 8}{1000ms} = 163.84kbps$

*附录： 第三章你或许应该知道的名词缩写

-
- **GBN**: Go Back N, 回退N步
 - **SR**: Selective Repeat, 选择重传
 - **ACK**: Acknowledgment, 确认
 - **RDT**: Reliable Data Transfer protocol, 可靠数据传输协议
 - **FSM**: Finite-State Machine, 有限状态自动机
 - **MSS**: Maximum Segment Size, 最大报文段长度
 - **MTU**: Maximum Transmission Unit, 最大传输单元
 - **RTT**: Round Trip Time, 往返时间
 - **SYN**: Synchronize Sequence Number, 同步序列编号
 - **AIMD**: Additive-Increase, Multiplicative-Decrease, 加性增, 乘性减
 - **ssthresh**: slow start threshold, 慢启动阈值
 - **cwnd**: congest window, 拥塞窗口
 - **rwnd**: receive window, 接收窗口